RESEARCH-ARTICLE

PEAK: Cost-Adaptive Profiling in a Heartbeat

YUHENG CHEN, The University of Texas at Austin, Austin, TX, United States

JUNJIE LI, The University of Texas at Austin, Austin, TX, United States

CHUN YAUNG LU, The University of Texas at Austin, Austin, TX, United States

YINZHI WANG, The University of Texas at Austin, Austin, TX, United States

Published: 16 November 2025

SC Workshops '25: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis
November 16 - 21, 2025
MO, St Louis, USA

Conference Sponsors:
SIGHPC

PEAK: Cost-Adaptive Profiling in a Heartbeat

Yuheng Chen
Texas Advanced Computing Center
The University of Texas at Austin
Austin, USA
yuheng.chen@utexas.edu

Junjie Li
Texas Advanced Computing Center
The University of Texas at Austin
Austin, USA
nicejunjie@gmail.com

Chun-Yaung Lu
Texas Advanced Computing Center
The University of Texas at Austin
Austin, USA
alu@tacc.utexas.edu

Yinzhi Wang
Texas Advanced Computing Center
The University of Texas at Austin
Austin, USA
yinzhi.wang.cug@gmail.com

Abstract

Instrumentation-based profiling is essential for uncovering fine-grained optimization opportunities in High-Performance Computing (HPC) and cloud applications, yet static instrumentation methods often impose fixed profiling overheads that cannot adapt to the dynamic workloads from applications at runtime. We further develop PEAK, a Dynamic Binary Instrumentation (DBI)-based profiler with two complementary modes for overhead control: a static mode, which enforces an upper limit on the absolute instrumentation overhead, and a dynamic mode based on the heartbeat mechanism, which controls the relative overhead in real time to maintain a user-defined ratio. Evaluations of workloads ranging from compute-intensive kernels to lightweight functions show that the heartbeat mechanism effectively bounds overhead while improving profile accuracy compared to static methods, delivering predictable and adaptive profiling performance for long-running, dynamic workloads.

CCS Concepts

• **Software and its engineering → Software performance**; • **General and reference → Performance**.

Keywords

profiling, application performance, system tools

ACM Reference Format:
Yuheng Chen, Junjie Li, Chun-Yaung Lu, and Yinzhi Wang. 2025. PEAK: Cost-Adaptive Profiling in a Heartbeat. In *Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops '25), November 16–21, 2025, St Louis, MO, USA.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3731599.3767521

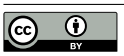
1 Introduction

High-Performance Computing (HPC) is a cornerstone of modern scientific research, enabling large-scale simulations, complex modeling, and big data analysis. As the demand for computational power pushes systems from the petascale to the exascale, the end of traditional performance scaling described by Moore's Law has placed a new emphasis on software efficiency. In this context, performance profiling has emerged as an important area for maximizing the performance of complex parallel applications and the powerful hardware on which they run. HPC profiling tools provide a granular view into the runtime behavior of applications on parallel architectures. They are instrumental in identifying performance bottlenecks, communication inefficiencies, and suboptimal memory access patterns that can limit scalability and waste valuable computational resources. By providing detailed, actionable insights, these tools empower developers, scientists, and engineers to precisely target and resolve performance issues, thereby optimizing the code and accelerating scientific discovery.

Although sampling-based profilers are effective in identifying high-level performance trends, they often lack the granularity required for detailed optimization. For this level of insight, instrumentation is essential. Techniques such as Dynamic Binary Instrumentation (DBI) are particularly powerful, as they allow for the precise measurement of selected functions even without access to the application's source code. However, the primary challenge for existing instrumentation-based tools is managing the runtime overhead they introduce. This overhead can become prohibitive in long-running applications, potentially altering the program's performance characteristic and making the collected data unrepresentative.

This problem is exacerbated by the inflexibility of traditional static profiling strategies. These approaches often impose a fixed limit on the number of instrumented calls; once the limit is reached, all profiling ceases. Such a rigid methodology is ill-suited for applications with dynamic behavior, as it cannot adapt to shifting computational phases. Consequently, this can lead to excessive and overwhelming data collection during compute-intensive regions while providing insufficient data during other critical phases, ultimately hindering a comprehensive performance analysis. This limitation highlights the need for a more adaptive and intelligent instrumentation framework.

We build on the PEAK[7, 12] profiler, a DBI-based function-level profiler designed to deliver high-resolution performance insights with adaptive overhead. Here we introduce two key mechanisms to PEAK: *runtime detachment*, which restores instrumentation targets to its original state once a predefined overhead budget is reached,



SC Workshops '25, St Louis, MO, USA

ACM ISBN 979-8-4007-1871-7/25/11
https://doi.org/10.1145/3731599.3767521

and the *Heartbeat* mechanism, which periodically reattaches the instrumentation targets based on real-time overhead measurements. Together, these enable cost-adaptive profiling, maintaining a user-defined relative overhead bound while preserving profiling accuracy.

Our contributions include:

(1) A DBI-based profiler that supports the detachment and reattachment of instrumentation at runtime on demand.
(2) An adaptive overhead control policy to manage the accuracy-overhead trade-off in real-time during profiling.
(3) Quantitative analysis of parameters and their impact on profiling accuracy and overhead.

By combining fine-grained function-level measurement with adaptive overhead control, PEAK bridges the gap between detailed performance analysis and practical runtime constraints, making it particularly effective for long-running or dynamic applications.

2 Background

2.1 Existing Profiling Tools

There are several profiling tools commonly used on HPC systems, each designed to analyze and optimize different facets of program performance. The primary distinction among these tools lies in their data collection methodology: sampling or instrumentation.

Sampling is a statistical approach that gathers performance data by periodically interrupting a program's execution to record its state, such as the program counter or call stack. This method is relatively non-intrusive as it does not require modifications to the source code, resulting in low performance overhead. However, because of its periodic nature, it may fail to capture short-lived events, potentially leading to an incomplete performance picture. Prominent profilers that utilize sampling include HPCtoolkit[1] and Linaro MAP[4].

In contrast, instrumentation involves inserting measurement probes directly into the program's source code or binary. These probes precisely record specific events, such as function calls or message passing operations, as they occur. While this technique provides highly detailed and accurate performance data, the added instructions can introduce non-trivial overhead, potentially perturbing the program's natural execution behavior. Widely-used instrumentation-based profilers include TAU (Tuning and Analysis Utilities)[10] and Score-P[6].

Furthermore, some tools employ a hybrid approach, combining both sampling and instrumentation to offer comprehensive analysis capabilities. For instance, Intel VTune Profiler[9] leverages both techniques to provide a holistic view of application performance, from high-level statistical analysis to detailed event traces.

2.2 Dynamic Binary Instrumentation for Function-Level Profiling

Dynamic Binary Instrumentation (DBI) [5] provides a compelling alternative by enabling runtime code modification without access to the original source code. DBI works by inserting instrumentation code directly into compiled binaries, allowing profiling and control at function granularity during execution. A core technique in DBI is the *trampoline* [3], which modifies the entry point of a target function to jump to a custom instrumentation routine, and then returns control to the original execution flow. Intercepting a function with a trampoline typically involves: (1) allocating executable memory for the trampoline code, (2) patching the first few instructions of the target function with a jump, (3) invoking the instrumentation logic, and (4) restoring execution to the original code.

Several DBI frameworks are available, including Frida [8], Dyninst [2], DynamoRIO [3], and PIN [5]. While DBI has been widely used in debugging, reverse engineering, and program analysis, applying it to continuous performance profiling in HPC environments presents unique challenges. In particular, frequent instrumentation of high-cost or frequently executed functions (e.g., BLAS or LAPACK kernels) can incur substantial runtime overhead. Therefore, making DBI practical for online performance profiling requires novel mechanisms to balance profiling accuracy with acceptable runtime cost.

The PEAK system builds upon DBI to enable function-level profiling with adaptive overhead control. In particular, it integrates runtime detachment and the Heartbeat mechanism to dynamically attach or remove instrumentation based on measured profiling cost. This design directly addresses the limitations of traditional offline profiling, providing a foundation for cost-adaptive, real-time performance monitoring with instrumentation in production HPC and cloud environments.

3 The PEAK Profiler

PEAK incorporates two distinct mechanisms for managing profiling overhead: a static cost-based limiter and a dynamic, adaptive Heartbeat mechanism.

3.1 Static Control: The PEAK_COST Mechanism

Static profiling control in PEAK, provided by the *PEAK_COST* mechanism, limits the total allowable profiling *overhead* through a fixed cost budget. The budget is derived from the measured per-call *overhead* of the target function and translated into a maximum number of instrumented calls. Once this limit is reached, the profiler detaches, ensuring predictable *overhead* control in workloads with stable performance characteristics.

3.1.1 Key Profiler Parameters.

PEAK_COST. This parameter is a user-defined threshold that controls the maximum cumulative overhead the profiler is allowed to incur before automatically detaching from the target function. The number of calls (N) the profiler will record before detaching is determined by the formula:

$$N = PEAK\_COST / peak\_general\_overhead. \quad (1)$$

This design allows users to specify an acceptable overhead budget in terms of time rather than a fixed number of calls. This mechanism provides users with a direct means to cap the total profiling overhead, ensuring that profiling activities do not disproportionately degrade application performance. This is a core advantage of the design of PEAK.

Peak General Overhead. This parameter quantifies the per-call base overhead introduced by PEAK. This value is determined during the initialization phase. Its measurement involves attaching the profiler to an empty function and comparing its execution time against a baseline execution time of the same empty function without profiling. The difference in execution times, divided by the number of calls, yields the peak_general_overhead. This ensures the measurement represents the minimal, inherent cost of instrumentation itself, independent of the target function's logic. This is a fixed, inherent cost per instrumentation event, determined by the profiler's internal implementation and underlying system characteristics, and is not influenced by *PEAK_COST*. Thus, while *PEAK_COST* can cap the total overhead, it cannot reduce this fundamental per-call cost, making it an aspect of the profiler not directly controllable by parameter tuning.

3.1.2 Metrics and Calculation Formulas. Two primary metrics were used to evaluate the performance of PEAK via *PEAK_COST*: Accuracy and Overhead.

Accuracy. Accuracy quantifies the completeness of the profiling data, representing the ratio of successfully profiled function calls to the total number of calls to the target function. It is defined as

$$Accuracy = \frac{Number\ of\ Profiled\ Calls}{Total\ Number\ of\ Calls}. \tag{2}$$

Higher accuracy implies a more complete capture of the target function's execution behavior, which is crucial for detailed performance analysis. However, results indicate that high accuracy often comes with higher overhead.

Overhead. Overhead measures the additional time introduced by the profiling activity, expressed as a percentage relative to the application's baseline execution time.

Absolute Overhead. The general formula provided by the user is

$$\left\lceil \frac{sum\_num\_calls}{thread\_count} \right\rceil \times peak\_general\_overhead. \tag{3}$$

For this single-threaded experiment, $thread\_count = 1$, so the formula simplifies to

$$Absolute\ Overhead = sum\_num\_calls \times peak\_general\_overhead. \tag{4}$$

Percentage Overhead. The relative overhead with respect to total execution time is given by

$$Percentage\ Overhead = \frac{Absolute\ Overhead}{Total\ Execution\ Time}. \tag{5}$$

Minimizing overhead is crucial for practical profiling, as excessive overhead can distort performance measurements or make profiling impractical in production systems.

3.2 Dynamic Control: The Heartbeat Mechanism

Traditional static profiling methods, such as the previously discussed *PEAK_COST* mechanism, provide fixed control over profiling duration or call counts. However, their main limitation lies in the inability to dynamically adjust profiling behavior based on the program's runtime characteristics. This rigidity often results in excessive overhead during critical execution phases or insufficient profiling data during low-activity periods, thereby reducing both the accuracy and efficiency of profiling in long-running or performance-variable applications.

To overcome these limitations, PEAK incorporates a Heartbeat mechanism that enables real-time, adaptive control of profiling overhead. Figure 1 shows the overall design of the Heartbeat Mechanism in PEAK. It consists of three components: 1) main profiling process (blue), 2) overhead control (yellow), and 3) accuracy control (green).

This mechanism runs on an independent thread that periodically wakes to evaluate the current profiling impact by computing an internal metric called the *current_profile_ratio*. This ratio dynamically reflects the total perceived overhead, incorporating both the accumulated profiling overhead and the Heartbeat mechanism's own costs.

Formally, the *current_profile_ratio* is calculated as:

$$current\_profile\_ratio = \frac{sum\_num\_calls \times peak\_general\_overhead}{thread\_count \times total\_execution\_time} + \frac{reattach\_overhead}{total\_execution\_time} \tag{6}$$

where *sum_num_calls* is the number of profiled function calls, *peak_general_overhead* is the average overhead per profiled call determined during the initialization phase, and *reattach_overhead* represents additional overhead specifically incurred by the Heartbeat mechanism itself. The parameter *thread_count* denotes the number of threads executing the profiled application, while the denominator *total_execution_time* accounts for the entire runtime duration of the profiled application.

For this single-threaded experiment, $thread\_count = 1$, so the formula simplifies to

$$current\_profile\_ratio = \frac{num\_calls \times peak\_general\_overhead}{total\_execution\_time} + \frac{reattach\_overhead}{total\_execution\_time} \tag{7}$$

This ratio serves as the control signal within the Heartbeat's adaptive loop. When the *current_profile_ratio* exceeds a predefined *target_profile_ratio*, the Heartbeat mechanism dynamically detaches the binary instrumentation hooks from the target functions to reduce overhead. Conversely, if the profiler is detached and the *current_profile_ratio* falls below the *target_profile_ratio* after a specified *hibernation_cycle*, the mechanism will only reattach the instrumentation when the ratio rises above the target again. This reattachment involves pausing other threads, executing an interceptor transaction to re-enable hooks, and then resuming paused threads.

3.2.1 Key Profiler Parameters.

Target Profile Ratio. This parameter defines the desired upper limit for profiling overhead, expressed as a ratio of the accumulated profiling cost to the total application execution time. It serves as the primary control knob for balancing the integrity of profiling data (*Accuracy*) and the performance impact (*Overhead*).

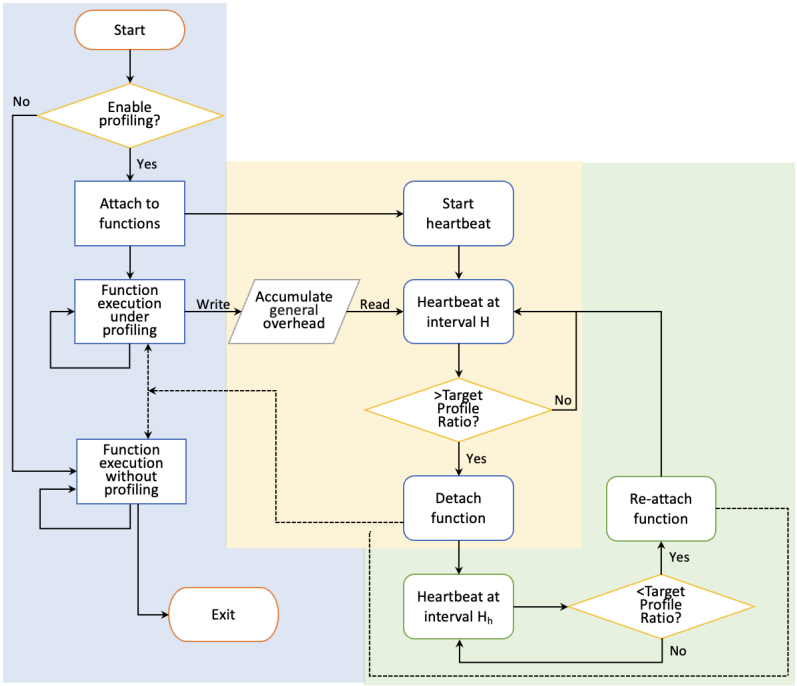


Figure 1: The workflow of the Heartbeat Mechanism in PEAK. H **is** *heartbeat_interval*, **and** H_h **is** *heartbeat_interval* × *hibernation_cycle*.

The Heartbeat mechanism continuously calculates an internal *current_profile_ratio*, and if this internal ratio exceeds the *target_profile_ratio*, detachment of instrumentation is initiated.

The *target_profile_ratio* allows users to explicitly specify their tolerance for profiling overhead. A higher ratio permits more extensive profiling (potentially leading to higher accuracy) at the cost of increased performance perturbation; conversely, a lower ratio prioritizes minimal impact on application performance. This ratio-based nature, rather than a fixed count, is precisely what enables dynamic adaptation to varying execution times and call costs.

Heartbeat Interval. This parameter dictates the frequency, in seconds, at which the dedicated Heartbeat thread wakes up to evaluate the *current_profile_ratio* and make detachment or reattachment decisions. In this experiment, *heartbeat_time* was fixed at 0.00005 seconds (50 microseconds).

A smaller *heartbeat_interval* allows for more rapid responses to changes in profiling overhead or application workload, enabling quicker adaptation. However, it can introduce its own overhead due to more frequent context switching and execution of the profiling logic, albeit a small one, which might affect the Heartbeat thread's own efficiency. The choice of *heartbeat_interval* represents a critical design trade-off between dynamic control responsiveness and the overhead of the profiling thread itself.

Reattachment Hibernation Cycle. This parameter controls how often the Heartbeat mechanism, while in a detached state, attempts to re-evaluate the *current_profile_ratio* to determine if reattachment is warranted. In this experiment, *hibernation_cycle* was fixed at 1000. This means a reattachment check occurs every 1000 Heartbeat profiling cycles.

This parameter is crucial for preventing continuous or overly frequent reattachment attempts, which could introduce unnecessary overhead and jitter. It ensures that the profiler only attempts to resume data collection after a sufficient period has passed and conditions may have improved, balancing the desire for comprehensive coverage with the cost of re-enabling instrumentation. The reattachment process itself, including thread pausing and transactional hook attachment, can be a non-negligible operation.

3.2.2 Metrics and Calculation Formulas. Two primary metrics were used to evaluate the performance of PEAK via Heartbeat: Accuracy and Overhead.

Accuracy. The definition of Accuracy is consistent with the description provided earlier in the text.

Overhead. Overhead measures the additional time introduced by the profiling activity, expressed both as an absolute time value and as a percentage relative to the application's baseline execution time.

General Overhead. The general overhead represents the extra time accumulated from instrumentation and profiling during function calls. It is calculated by the Equation 3. For this single-threaded experiment, *thread_count* = 1, so the formula simplifies to

$$\textit{General Overhead} = \textit{sum\_num\_calls} \times \textit{peak\_general\_overhead}. \quad (8)$$

Reattach Overhead. When a reattach event is triggered by the Heartbeat mechanism, all threads must be paused, causing additional overhead. The *Reattach Overhead* accounts for the total time spent during all such reattach operations, which must be included in the overall overhead calculation.

Percentage Overhead. The relative overhead with respect to the total execution time is given by

$$\text{Percentage Overhead} = \frac{\text{General Overhead}}{\text{Total Execution Time}} + \frac{\text{Reattach Overhead}}{\text{Total Execution Time}}. \tag{9}$$

Minimizing overhead is crucial for practical profiling, as excessive overhead can distort performance measurements or make profiling impractical in production systems.

4 Experimental Evaluation

All experiments were performed on the *Frontera* supercomputer [11]. The default software stack on the system was used, which is the GNU Compiler Collection (GCC) version 4.8.5 for all C, C++, and Fortran codes, and the MKL library version 19.1.1 for the dgemm function. The experiments were run on a single Cascade Lake compute node.

4.1 Performance of Static Control (PEAK_COST)

4.1.1 Experimental Setup. This experiment aims to observe the behavior of PEAK when analyzing a computationally intensive function under varying *PEAK_COST* values.

The `dgemm` function, a double-precision general matrix multiplication routine, was selected as the target function for profiling. This function was chosen due to its predictable and measurable execution characteristics, making it suitable for performance analysis. Here, we use `dgemm` with a matrix size of $M = N = K = 80$. We picked this matrix size by enumerating different values of N to compare the execution time and PEAK's *Overhead* such that the overhead is in the 5% to 10% range of the execution time. The dgemm function was invoked within a loop for a total of 100,000 iterations. So the total number of calls was fixed at 100,000. The *Number of Profiled Calls* corresponds to the *sum_num_calls* from the data shown in Table 1. The large number of calls ensures sufficient data points to observe PEAK's long-term behavior and detachment effects. The *PEAK_COST* parameter was systematically varied across a range from 0.075 to 0.4 (as shown in Table 1) to analyze its impact on profiling Accuracy and Overhead.

The experiment was conducted in a single-threaded environment, which simplifies the overhead analysis by eliminating complexities introduced by concurrent profiling or thread synchronization. The user-provided overhead calculation formula can be expressed as Equation 3. If *peak_general_overhead* represents the "per-call" overhead, the total overhead should simplify to

$$\textit{sum\_num\_calls} \times \textit{peak\_general\_overhead}. \tag{10}$$

For the provided formula to match this intuitive definition, it must hold that $\textit{thread\_count} = 1$. In this specific experiment, where the C code is single-threaded, $\lceil \textit{sum\_num\_calls}/1 \rceil$ reduces to *sum_num_calls*. This assumption clarifies the interpretation of the overhead metric as the direct product of the number of profiled calls and the per-call overhead, avoiding ambiguities related to multi-threaded overhead accounting that are outside the scope of the current dataset. It also suggests that the *thread_count* term in the general formula supports broader profiler capabilities, but is unused in this experiment.

Table 1: Effect of PEAK_COST

PEAK_COST	Calls	Acc.(%)	AbsOH(s)	OH(%)
0.075	25571	25.57%	0.075	3.11%
0.100	33496	33.50%	0.100	4.11%
0.125	42575	42.58%	0.125	5.06%
0.150	51367	51.37%	0.150	5.98%
0.175	58642	58.64%	0.175	6.93%
0.200	67203	67.20%	0.200	7.76%
0.225	77056	77.06%	0.225	8.59%
0.250	85499	85.50%	0.250	9.39%
0.275	90486	90.49%	0.275	10.28%
0.300	100000	100.00%	0.295	10.81%
0.400	100000	100.00%	0.295	10.96%

4.1.2 Results and Discussion. The experimental results are summarized in Table 1, illustrating the interplay between *PEAK_COST*, Accuracy, and Overhead.

Impact of PEAK_COST on Accuracy. As *PEAK_COST* increases from 0.075 to 0.3, *Accuracy* consistently rises from 25.57% to 100%. This trend is a direct consequence of the relationship allows more calls to be profiled before the detachment threshold is reached. Once *PEAK_COST* exceeds 0.3, *Accuracy* remains at 100%, indicating that at these higher *PEAK_COST* values, PEAK is able to complete profiling all 100,000 calls before the accumulated overhead reaches the *PEAK_COST* limit.

Impact of PEAK_COST on Overhead. For *PEAK_COST* values below approximately 0.295 seconds, the observed *Absolute Overhead* is directly equal to the *PEAK_COST* value. By comparing Table 1's *PEAK_COST* column with the *Absolute Overhead* column, it can be seen that when *Accuracy* is less than 100%, the *Absolute Overhead* exactly matches the *PEAK_COST* value. The design of PEAK explicitly states that *PEAK_COST* controls the maximum overhead allowed before detachment. The number of calls N is given by Equation 1. This implies that when the accumulated overhead reaches *PEAK_COST*, the profiler detaches. This direct correspondence confirms that for these lower *PEAK_COST* values, the profiler enforces *PEAK_COST* as a hard cap on the total overhead incurred. Profiling stops precisely when the overhead budget is exhausted, even if not all calls have been profiled. This demonstrates the direct controllability and effectiveness of the *PEAK_COST* parameter in limiting the profiling impact.

Once *PEAK_COST* reaches 0.3 and above, the *Absolute Overhead* stabilizes at approximately 0.295 seconds. For the values of 0.3 and 0.4, the *Absolute Overhead* is no longer equal to *PEAK_COST* but fluctuates around a fixed value ($\approx$ 0.295 seconds). Concurrently, *Accuracy* reaches 100%, meaning all 100,000 calls to `dgemm` have been profiled. At this point, the total overhead becomes

$$100{,}000 \times \textit{peak\_general\_overhead}. \tag{11}$$

Since *peak_general_overhead* $\approx 2.95 \times 10^{-6}$ seconds/call,

$$100{,}000 \times 2.95 \times 10^{-6} \approx 0.295 \text{ seconds.}$$

At these higher *PEAK_COST* values, *PEAK_COST* threshold (e.g., 0.3 seconds) is higher than the inherent total overhead required to profile all 100,000 calls (≈ 0.295 seconds). Therefore, the profiler never reaches its *PEAK_COST* limit and profiles the entire workload. *PEAK_COST* still acts as an upper bound, but it is no longer the limiting factor for the actual *Overhead*. This confirms that *PEAK_COST* provides a controllable upper bound, but the actual *Overhead* will be less than or equal to this bound, depending on the inherent total profiling cost of the workload.

Accuracy-Overhead Trade-off. The data reveals a clear trade-off: to achieve higher *Accuracy* (e.g., from 25.57% to 100%), a higher *PEAK_COST* must be tolerated, which in turn leads to a higher *Absolute Overhead*. Conversely, to minimize *Overhead*, *Accuracy* must be sacrificed. This fundamental trade-off is a common characteristic of performance profiling tools. As *PEAK_COST* increases, *Accuracy* generally improves (from 25.57% to 100%), and *Absolute Overhead* increases accordingly (from 0.075 s to approximately 0.295 s).

PEAK_COST directly determines *N* (the maximum number of profiled calls) via Equation 1. A higher *PEAK_COST* results in a larger *N*, allowing more calls to be profiled, which directly translates to higher *Accuracy*. An increase in *sum_num_calls* (due to higher *Accuracy*) directly leads to higher *Absolute Overhead*. This trend continues until *Accuracy* reaches 100% (i.e., *sum_num_calls* = 100,000). At this point, *sum_num_calls* cannot increase further, and thus *Absolute Overhead* also stabilizes, regardless of further increases in *PEAK_COST*.

This demonstrates a fundamental trade-off: achieving more complete analysis (higher *Accuracy*) necessarily incurs a greater performance penalty (higher *Overhead*) until all calls are profiled. The *PEAK_COST* parameter allows users to explicitly manage this trade-off, choosing a balance that aligns with their specific analysis goals and performance constraints.

4.1.3 Motivation: The Need for Adaptive Overhead Control. The effectiveness and intrusiveness of a profiler are heavily dependent on the execution characteristics of the target function. A one-size-fits-all profiling strategy is often suboptimal. To illustrate this, we consider two extreme scenarios that motivate the need for adaptive overhead control mechanisms.

Case 1: Target Function with Large Intrinsic Cost. In the first case, the profiled function is computationally expensive, such that the profiler's instrumentation overhead is negligible compared to the function's own execution time. We simulate this scenario using a `dgemm` kernel operating on large matrices with a matrix size of $M = N = K = 2000$, executed 100 times in a loop. The matrix data is dynamically allocated and initialized to avoid compiler optimizations, and each iteration performs a full matrix multiplication.

The experimental results are summarized in Table 2.

In this scenario, even with full instrumentation (*Accuracy* = 100%), the profiler-induced *Overhead* remains negligible relative to the function's intrinsic execution time. Therefore, for such high-cost functions, detaching the profiler provides little benefit and may unnecessarily reduce accuracy.

Table 2: PEAK_COST results for a target function with large intrinsic cost

PEAK_COST	Accuracy (%)	Overhead (%)
0.001	100.00%	0.00124%
0.0001	34.00%	0.00042%
0.000001	1.00%	0.000013%

Case 2: Target Function with Small Intrinsic Cost. In the second case, the target function is extremely lightweight, so the profiler's instrumentation overhead becomes significant relative to the function's own runtime. We simulate this scenario using a simple function that returns a random value and is invoked 10,000 times in a loop. This function performs only trivial computation, making its intrinsic cost orders of magnitude smaller than the instrumentation overhead.

The results are shown in Table 3.

Table 3: PEAK_COST results for a target function with small intrinsic cost

PEAK_COST	Accuracy	Overhead
0.000003	0.01%	2.67%
0.00003	0.1%	21.48%
0.003	10.27%	103.31%
0.3	100%	107.47%

Here, even a modest amount of instrumentation can more than double the function's runtime. For such low-cost functions, the profiler should only profile it once (or very few times) before detaching to minimize perturbation.

Implications for Overhead Control Mechanisms. These cases demonstrate that the optimal profiling strategy is highly dependent on the target function's intrinsic cost. An effective profiler must therefore adapt its behavior, which necessitates robust and flexible overhead control mechanisms.

4.2 Performance of Dynamic Control (Heartbeat)

4.2.1 Experimental Setup. This experiment aims to investigate the relationship between the *target_profile_ratio* and the resulting profiling *Accuracy* and *Overhead*, demonstrating how the Heartbeat mechanism enables dynamic and controllable overhead, thereby facilitating PEAK's potential evolution into a continuous monitoring solution.

The `dgemm` function was chosen as the target for profiling. We used the same matrix size as in the previous PEAK_COST experiments ($M = N = K = 80$). In contrast to the *PEAK_COST* experiment, the total number of `dgemm` calls in this experiment was significantly increased to 3,000,000 iterations to better observe the long-term adaptive behavior of the Heartbeat mechanism. The large number of iterations ensures that the Heartbeat mechanism has sufficient time to engage its dynamic control, which are crucial

Table 4: Effect of Target Profile Ratio

Ratio	Calls	Acc.(%)	General OH (s)	Reattach OH (s)	Abs OH (s)	OH (%)	Reattach Cnt
0.04	199245	6.64%	0.594	1.405	2.000	4.00%	288
0.05	375495	12.52%	1.083	1.426	2.509	4.99%	292
0.06	309583	10.31%	0.908	2.104	3.012	5.99%	427
0.07	493090	16.44%	1.442	2.112	3.554	6.99%	429
0.08	893619	29.79%	2.547	1.546	4.093	7.99%	316
0.09	1108761	36.96%	3.169	1.475	4.644	9.00%	302
0.10	1233728	41.12%	3.660	1.496	5.156	9.99%	307
0.11	1531138	51.04%	4.414	1.375	5.789	11.00%	282

for assessing its long-term monitoring capabilities. It ensures that the *peak_general_overhead* per call is a small but measurable fraction of the total execution time, making the *target_profile_ratio* a meaningful control parameter.

The primary independent variable in this experiment was the *target_profile_ratio*. Its value was systematically varied across different experimental runs to observe its impact on profiling *accuracy* and the resulting *overhead*. This allowed for the exploration of the trade-off curve between these two critical metrics. Across all experimental runs, *heartbeat_interval* was kept constant at 0.00005 seconds, and *hibernation_cycle* was fixed at 1000. The reattachment mechanism was consistently enabled for all runs.

To ensure that the observed effects were solely attributable to the Heartbeat mechanism, the *PEAK_COST* control was effectively disabled during these experiments. This was achieved by setting the *PEAK_COST* parameter to an extremely large value, ensuring that the profiler would not detach based on a fixed call count limit. This isolation is a critical aspect of the experimental design, allowing for a clear and unambiguous evaluation of the Heartbeat's dynamic control capabilities without confounding factors from the static *PEAK_COST* mechanism.

4.2.2 Results and Discussion. The experimental results are summarized in Table 4, illustrating the interplay between *target_profile_ratio*, Accuracy, and Overhead.

Accuracy and Overhead Trends. As presented in Table 4, increasing the *target_profile_ratio* consistently leads to an increase in both profiling *accuracy* and the resulting *overhead*. For example, when the *target_profile_ratio* was set to 0.07, the observed *accuracy* was 16.44% with a corresponding *overhead* of 6.99%. Increasing the *target_profile_ratio* to 0.11 resulted in a significantly higher *accuracy* of 51.04%, accompanied by an *overhead* of 11%. This positive correlation is expected: a higher allowed overhead ratio permits the profiler to remain attached for longer durations, thereby capturing more function calls.

This trend reflects the fundamental trade-off inherent in the Heartbeat mechanism: achieving higher profiling completeness (*Accuracy*) necessarily demands greater performance perturbation (*Overhead*). By adjusting the target ratio, the user directly controls this trade-off.

A key observation is that the measured *overhead* consistently remains close to the specified *target_profile_ratio*, validating the mechanism's ability to bound profiling costs. For instance, with a *target_profile_ratio* of 0.10, the actual measured overhead was 9.99%, demonstrating the robustness of the *current_profile_ratio* calculation and detachment logic in enforcing the performance constraint.

Ratio–Interval–Reattach Interplay. An interesting anomaly occurs at *target_profile_ratio* = 0.06: the number of *reattach* events spikes sharply to 427, compared to 292 at 0.05 and 316 at 0.08. This is a critical-point phenomenon, caused by the interaction between three factors:

(1) **Steady-State Attached-Mode Overhead:** When the profiler remains continuously attached under a fixed workload, the actual overhead ratio tends to converge to a relatively stable value determined by the call rate, the per-call overhead (*peak_general_overhead*), and the constant *heartbeat* cost. We refer to this stable value as the *steady-state attached-mode overhead*.
(2) **Threshold Placement Relative to the Steady State:** When the *target_profile_ratio* is set well below the steady-state attached-mode overhead, the profiler will quickly detach after exceeding the threshold, and then must remain detached for a relatively long time while the cumulative ratio decays. This produces fewer reattachments (as seen at 0.05). Conversely, when the threshold is well above the steady-state overhead, the profiler stays attached for most of the run (as at 0.08 or higher), again producing few reattachments. At 0.06, however, the threshold is very close to the steady-state overhead, causing the control loop to oscillate around the boundary: small timing variations, the discrete nature of sampling, and the instantaneous cost of reattachment frequently push the ratio above and below the threshold, triggering rapid attach/detach cycles (chattering).
(3) **Discrete Sampling and Reattach Cost:** The *reattach* decision is evaluated only at intervals determined by *hibernation_cycle*, not continuously. Even when the ratio drops below the threshold, reattachment waits until the next sampling point. Upon reattachment, the immediate cost increases the overhead ratio, which can force another detach in the next cycle—amplifying oscillations.

This analysis highlights the strong coupling between *target_profile_ratio* and *hibernation_cycle*:

- **Lower ratio + short interval** → highly sensitive, prone to frequent switching.

- **Lower ratio + long interval** → fewer switches, but potential overshoot in overhead.
- At critical points such as *target_profile_ratio* = 0.06, where reattachment frequency spikes, increasing the hibernation cycle can effectively reduce the number of reattachments and thus lower the overall profiling overhead.

Comparison with PEAK_COST. The fundamental difference between *PEAK_COST* and the Heartbeat mechanism lies in their control principle and adaptability. *PEAK_COST* enforces a hard limit on the total number of profiled calls: once the predefined budget is exhausted, the profiler detaches and remains inactive for the rest of the execution. This is a one-way decision — the profiler will not reattach, regardless of whether the application enters a phase where profiling could be performed at low cost. While this static cap can prevent uncontrolled profiling, it cannot guarantee a specific percentage of *overhead*, and it often either under-utilizes available profiling opportunities or overburdens the application during expensive phases.

In contrast, the Heartbeat mechanism functions as a continuous feedback loop. It profiles the real-time ratio and dynamically attaches or detaches the profiler to keep the total *overhead* within the user-defined bound. Even after detachment, it continues tracking call counts and execution time, allowing reattachment when the measured ratio drops below the target. This gives PEAK the ability to adaptively trade accuracy for overhead throughout the run: a higher *target_profile_ratio* yields more frequent profiling (higher accuracy at higher cost), while a lower ratio enforces stricter cost limits (lower overhead but lower accuracy).

This dynamic profiling capability turns PEAK from a simple budget enforcer into an adaptive profiling controller, able to take advantage of low-cost phases without exceeding the desired performance impact. Such adaptability is a critical distinction that will also be emphasized in our later comparison with TAU, where the Heartbeat mechanism's responsiveness to runtime conditions becomes even more apparent.

5 Multi-threaded Evaluation

5.1 Experimental Setup

To assess the scalability of PEAK's Heartbeat mechanism under multi-threaded workloads, we extended the single-threaded `dgemm` benchmark with OpenMP parallelism. Each thread executed 3,000,000 calls to the target function, using private output buffers to avoid data races and a shared read-only matrix for input.

We tested three configurations with 2, 28, and 56 threads. For the 2- and 28-thread cases, the Heartbeat parameters were set to *heartbeat_interval* = 0.1, *target_profile_ratio* in the range [0.04, 0.11], and *hibernation_cycle* = 50, with the chosen *target_profile_ratio* range matching that of the single-threaded Heartbeat experiments in Section 4.2. For the 56-thread case, *heartbeat_interval* was relaxed to 0.2 while keeping other parameters unchanged.

5.2 Results and Discussion

Tables 5, 6, and 7 summarize the measured profiling *Accuracy* and *Overhead* for the three thread counts. For 2 threads, the accuracy grows steadily with higher target ratios (6.59% at 0.04 to 17.27% at 0.11), while the overhead rises proportionally (3.81%–9.54%). At 28 threads, accuracy is lower in absolute terms (0.91%–4.88%), reflecting the increased volume of calls per unit time, but overhead remains well-controlled (4.14%–10.82%). At 56 threads, further relaxation of the interval yields stable overhead (3.61%–9.28%), with accuracy scaling up to 4.04%.

These results demonstrate that the Heartbeat mechanism preserves its ability to bound overhead even under substantial thread-level parallelism, though profiling coverage naturally decreases as thread count increases. The reduction in accuracy with higher thread counts is expected, as each detach and reattach operation requires interrupting the execution of all threads, which increases both the *reattach_overhead* and the *peak_general_overhead*.

According to Equation 6, the additional *reattach_overhead* and *peak_general_overhead* lead to a larger overall ratio compared to fewer threads. Consequently, for the same *target_profile_ratio*, executions with higher thread counts reach the detachment threshold sooner, resulting in fewer profiled calls and reduced accuracy.

Nevertheless, the stability of the overhead across 2, 28, and 56 threads indicates that the Heartbeat mechanism scales predictably with multiple threads, avoiding uncontrolled cost escalation. This property is particularly important in HPC environments, which often employ hybrid parallelism with many MPI processes and multiple threads per process. Because the Heartbeat mechanism operates independently within each process, its overhead control scales naturally to multi-process, multi-threaded environments: as long as stability is ensured in the multi-threaded case, the same guarantee extends across multi-process executions.

Table 5: Multi-threaded Heartbeat results with 2 threads

Ratio	Accuracy	Overhead
0.04	6.59%	3.81%
0.05	7.45%	4.29%
0.06	9.05%	5.10%
0.07	10.56%	5.94%
0.08	12.22%	6.93%
0.09	14.22%	7.88%
0.10	15.41%	8.63%
0.11	17.27%	9.54%

Table 6: Multi-threaded Heartbeat results with 28 threads

Ratio	Accuracy	Overhead
0.04	0.91%	4.14%
0.05	1.20%	4.65%
0.06	1.94%	5.85%
0.07	2.50%	6.84%
0.08	3.17%	7.90%
0.09	3.63%	8.69%
0.10	4.31%	9.75%
0.11	4.88%	10.82%

Table 7: Multi-threaded Heartbeat results with 56 threads

Ratio	Accuracy	Overhead
0.04	0.59%	3.61%
0.05	1.16%	4.62%
0.06	1.79%	5.56%
0.07	2.14%	6.02%
0.08	2.73%	7.01%
0.09	3.09%	7.75%
0.10	3.44%	8.34%
0.11	4.04%	9.28%

6 Comparison with TAU Profiler

To further evaluate the practicality of PEAK in controlling profiling *Accuracy* and *Overhead*, we compare it with the widely used TAU performance analysis toolkit [10]. Unlike the Heartbeat experiments in Section 4.2, this comparison introduces a lightweight `dgemm_wrapper` function around the `dgemm` call. This wrapper is necessary because TAU cannot directly instrument functions from external libraries without source-level integration; instead, it operates on user-defined functions.

The TAU is pre-installed on Frontera as a module using the Intel 19.1.1 compiler. For fair comparison, both PEAK and TAU experiments below were compiled using the same Intel compiler toolchain. In the PEAK experiments, we set *target_profile_ratio* to 0.05, *heartbeat_interval* to 0.00005, and *hibernation_cycle* to 1000. Three workloads were tested, with the target function invoked 2,000,000, 3,000,000, and 4,000,000 times, respectively.

The results for PEAK are shown in Table 8, demonstrating that both *Accuracy* and *Overhead* remain stable across varying call counts.

Table 8: PEAK results for wrapper-instrumented `dgemm` under varying call counts

Calls	Accuracy	Overhead
2,000,000	12.60%	5.00%
3,000,000	12.56%	5.00%
4,000,000	12.54%	5.00%

For TAU, we used its `throttle` mechanism, which imposes a static limit on the number of instrumented calls. To match the first workload's call count, the `throttle` limit was set to 252,044 calls, resulting in the same number of recorded calls (252,045) in the 2,000,000 workload—exactly matching the number of calls profiled by PEAK in the same setting.

However, as shown in Table 9, *Accuracy* decreases significantly as the total call count increases, illustrating that TAU's static limit mechanism cannot maintain consistent profiling ratios across workloads.

These results highlight a key distinction: while TAU's `throttle` can control the upper limit of instrumentation overhead, it lacks the adaptability of PEAK's Heartbeat mechanism, which dynamically maintains a user-defined profiling ratio regardless of workload size.

Table 9: TAU results for wrapper-instrumented `dgemm` under varying call counts

Calls	Accuracy	Overhead
2,000,000	12.60%	1.90%
3,000,000	8.40%	0.50%
4,000,000	6.30%	0.30%

Also, TAU incurs lower profiling overhead due to its throttle using a lightweight implementation that does not fully detach the instrumentation code from the target application. PEAK, on the other hand, restores the target function to its original state, ensuring the program runs without profiler influence when inactive. The difference in profiling overhead is expected and demonstrated previously[12].

Nevertheless, TAU remains a highly capable performance analysis suite, offering a comprehensive set of features, including hardware counter integration, and call-path profiling that extend beyond the targeted function-level profiling focus of PEAK.

7 Conclusion

We further developed PEAK, a DBI-based profiler for cost-adaptive function-level profiling in HPC and cloud applications. By extending a static mode into a dynamic mode with the Heartbeat mechanism, PEAK provides both predictable overhead budget enforcement and adaptive overhead control, enabling continuous profiling without exceeding user-defined overhead limits. Experimental evaluation on workloads ranging from compute-intensive kernels to lightweight functions demonstrated that Heartbeat can maintain profiling overhead close to the target ratio while significantly improving accuracy compared to static methods. These results highlight PEAK's effectiveness in delivering fine-grained, low-intrusion performance profiling for long-running, dynamic applications, offering a practical approach for bridging the gap between detailed offline profiling and coarse-grained performance monitoring in production environment.

Acknowledgments

This work is supported by the National Science Foundation through the OAC-2402542 award. The authors also acknowledge AI tools including Gemini, ChatGPT, and Claude that helped revise the manuscript.

References

[1] L. Adhianto, S. Banerjee, M. Fagan, M. Krentel, G. Marin, J. Mellor-Crummey, and N. R. Tallent. 2010. HPCTOOLKIT: tools for performance analysis of optimized parallel programs http://hpctoolkit.org. *Concurr. Comput. : Pract. Exper.* 22, 6 (April 2010), 685–701.

[2] Andrew R. Bernat and Barton P. Miller. 2011. Anywhere, any-time binary instrumentation. In *Proceedings of the 10th ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools* (Szeged, Hungary) *(PASTE '11)*. Association for Computing Machinery, New York, NY, USA, 9–16. doi:10.1145/2024569.2024572

[3] Derek Bruening, Timothy Garnett, and Saman Amarasinghe. 2003. An infrastructure for adaptive dynamic optimization. In *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization* (San Francisco, California, USA) *(CGO '03)*. IEEE Computer Society, USA, 265–275.

[4] Christopher January, Jonathan Byrd, Xavier Oró, and Mark O'Connor. 2015. Allinea MAP: Adding Energy and OpenMP Profiling Without Increasing Overhead. In *Tools for High Performance Computing 2014*, Christoph Niethammer, José Gracia, Andreas Knüpfer, Michael M. Resch, and Wolfgang E. Nagel (Eds.). Springer International Publishing, Cham, 25–35.

[5] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. 2005. Pin: building customized program analysis tools with dynamic instrumentation. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation* (Chicago, IL, USA) *(PLDI '05)*. Association for Computing Machinery, New York, NY, USA, 190–200. doi:10.1145/1065010.1065034

[6] Dieter an Mey, Scott Biersdorf, Christian Bischof, Kai Diethelm, Dominic Eschweiler, Michael Gerndt, Andreas Knüpfer, Daniel Lorenz, Allen Malony, Wolfgang E. Nagel, Yury Oleynik, Christian Rössel, Pavel Saviankou, Dirk Schmidl, Sameer Shende, Michael Wagner, Bert Wesarg, and Felix Wolf. 2012. Score-P: A Unified Performance Measurement System for Petascale Applications. In *Competence in High Performance Computing 2010*, Christian Bischof, Heinz-Gerd Hegering, Wolfgang E. Nagel, and Gabriel Wittum (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 85–97.

[7] peak team. 2024. *PEAK: Lightweight, versatile performance evaluation tool.* https://github.com/peak-team/peak

[8] Ole André V Ravnås. 2019. Frida: Dynamic instrumentation toolkit for developers, reverse-engineers, and security researchers.

[9] James Reinders. 2005. *VTune performance analyzer essentials: measurement and tuning techniques for software developers* (1. print ed.). Intel Press, Hillsboro, Or.

[10] Sameer S. Shende and Allen D. Malony. 2006. The Tau Parallel Performance System. *Int. J. High Perform. Comput. Appl.* 20, 2 (May 2006), 287–311. doi:10.1177/1094342006064482

[11] Dan Stanzione, John West, R. Todd Evans, Tommy Minyard, Omar Ghattas, and Dhabaleswar K. Panda. 2020. Frontera: The Evolution of Leadership Computing at the National Science Foundation. In *Practice and Experience in Advanced Research Computing 2020: Catch the Wave* (Portland, OR, USA) *(PEARC '20)*. Association for Computing Machinery, New York, NY, USA, 106–111. doi:10.1145/3311790.3396656

[12] Yinzhi Wang and Junjie Li. 2023. PEAK: a Light-Weight Profiler for HPC Systems. In *Proceedings of the SC '23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis* (Denver, CO, USA) *(SC-W '23)*. Association for Computing Machinery, New York, NY, USA, 677–680. doi:10.1145/3624062.3624143